ETSIINF (UPM)

Práctica 1 - SQLite y ORM

Asignatura: Bases de Datos II

Diego Mateo Fernández (diego.mateofer@alumnos.upm.es)

Anton Pogromskyy Mozharskyi (anton.pogromskyy@alumnos.upm.es)

Alejandro Escorial Aparicio (alejandro.escorial@alumnos.upm.es)

Estructura

En esta práctica separamos claramente las responsabilidades entre los archivos de código y la base de datos. Por tanto, contamos con 3 archivos principales.

main.py
Script principal que abre la conexión a *sakila.db*, crea la sesión de SQLAlchemy e implementa las funciones de cada ejercicio, lanzando consultas ORM y mostrando los resultados por consola.

models.py
Define las clases ORM (Customer, Rental, Film, Category, etc.) con sus columnas y relaciones, mapeando directamente las tablas reales de la base de datos Sakila.

sakila.db
Base de datos SQLite que contiene todos los datos del videoclub de ejemplo, sobre la que se ejecutan las consultas definidas en *main.py*.

Transformación de Sakila (MySQL --> SQLite)

Para realizar la traducción del volcado original de Sakila (MySQL) al formato SQLite, decidimos conectarnos a un servidor MySQL activo y extraer la información. Así evitamos la modificación manual de los archivos .sql.

Para no alterar nuestro entorno local y asegurar tanto la compatibilidad como la replicabilidad, levantamos un contenedor Docker temporal con la versión 9.0 de MySQL, exponiendo el puerto 3309 de nuestra máquina:

```
docker run --name mysql-sakila -e MYSQL_ROOT_PASSWORD=root -p 3309:3306 -d mysql:8.0
```

Una vez inicializado el servidor en segundo plano, inyectamos el esquema y los datos del volcado original de Sakila directamente en el contenedor:

```
docker exec -i mysql-sakila mysql -uroot -proot< sakila-schema.sql
docker exec -i mysql-sakila mysql -uroot -proot < sakila-data.sql
```

Para la traducción, utilizamos la librería de código abierto *mysql-to-sqlite3*, por lo que creamos un entorno virtual aislado de Python para su instalación:

```
python3 -m venv venv
source venv/bin/activate      <-- (para Mac y Linux)
pip install mysql-to-sqlite3
```

A diferencia de los scripts tradicionales que intentan limpiar el texto de un archivo .sql mediante expresiones regulares (lo cual generó errores de sintaxis en nuestro caso), *mysql-to-sqlite3* funciona conectándose directamente al motor MySQL que está en ejecución. La herramienta lee el esquema y los datos directamente de la memoria, traduce los tipos de datos a SQLite y escribe la información directamente en un nuevo archivo binario .db.

Mientras preparamos el entorno, nos encontramos con un error de importación (`ImportError: cannot import name 'Expression' from 'sqlglot'`):

```
Traceback (most recent call last):
  File "/Users/Anton/Downloads/sakila-db 2/venv/bin/mysql2sqlite", line 5, in <module>
    from mysql_to_sqlite3.cli import cli
  File "/Users/Anton/Downloads/sakila-db 2/venv/lib/python3.13/site-packages/mysql_to_sqlite3/__init__.py", line 5, in <module>
    from .transporter import MySQLtoSQLite
  File "/Users/Anton/Downloads/sakila-db 2/venv/lib/python3.13/site-packages/mysql_to_sqlite3/transporter.py", line 18, in <module>
    from sqlglot import Expression, exp, parse_one
ImportError: cannot import name 'Expression' from 'sqlglot' (/Users/Anton/Downloads/sakila-db 2/venv/lib/python3.13/site-packages/sqlglot/__init__.py). Did you mean: 'expressions'?
```

Lo resolvimos forzando la instalación de una versión específica y estable compatible con nuestro entorno:

```
pip install "sqlglot>=27.27.0,<30.0.0"
```

Finalmente, ejecutamos el comando final de extracción apuntando al puerto mapeado 3309:

```
mysql2sqlite -f sakila.db -d sakila -u root -p -h 127.0.0.1 -P 3309
```

Comentarios

Uno de los aspectos más importantes al realizar el diseño en *models.py* es que hemos definido únicamente lo estrictamente necesario. Para ello, en lugar de volcar toda la estructura de la base de datos Sakila y todos sus campos, decidimos definir únicamente las clases y las columnas necesarias para realizar las consultas requeridas. Si bien es cierto que la base de datos no es demasiado grande, esta medida nos permite optimizar el consumo de memoria y disminuir los tiempos de ejecución, además de mantener el código limpio y legible.

Otro aspecto importante es que, en lugar de utilizar consultas estáticas, varias de las funciones principales implementan consultas parametrizadas. De esta forma, funciones como las del ejercicio 3 o el 4 reciben parámetros externos directamente del usuario que modifican la consulta. Estos parámetros son tratados automáticamente de forma segura por SQLAlchemy, evitando problemas como la inyección SQL.

También es importante mencionar que la interacción con el usuario es gestionada a través de una interfaz en línea de comandos que funciona a modo de menú interactivo. Este menú es persistente y permite al usuario realizar múltiples operaciones en una misma sesión. Además, se tratan los posibles errores en la entrada proporcionada por el usuario y se definen valores predeterminados en caso de no ofrecer ninguna entrada.

Por último, cabe destacar que a la hora de realizar las consultas se hace uso de funciones nativas del motor de base de datos como por ejemplo *func.count*, *func.avg* o *func.max*.
Además, para las operaciones de escritura (ejercicios 7 y 8), hemos utilizado *session.flush()* (nos permite obtener los nuevos IDs en el ejercicio 8 antes de confirmar) y *session.commit()* para asegurar que los cambios se guardan en bloque.